Free-Networking-labs

操作マニュアル（管理者編）

サーバーの構築・運用・保守・トラブル対応

OSS版CML — containerlabで動くネットワーク演習環境

目次

1. はじめに 4

1.1 この文書について 4

1.2 システムの構成 4

1.3 ディレクトリとファイルの場所 6

2. 動作環境 6

メモリの目安（実測） 7

3. 新しいサーバーへのセットアップ 7

手順 1　OS と基本のパッケージ 7

手順 2　Docker 7

手順 3　clab-api-server 8

手順 3b　管理者を clab_admins に入れる 8

手順 4　Node.js とリポジトリ 9

手順 5　ノードのイメージ 9

手順 6　EdgeShark（キャプチャの裏方） 9

手順 7　プロキシ 3 つ（console-proxy・ovs-helper・capture-proxy） 9

手順 8　ovs-helper のラッパー（sudo の限定許可） 10

手順 9　画面のビルドと nginx 11

手順 10　ファイアウォール 11

手順 11　動作の確認 12

4. 利用者のアカウント管理 12

4.1 追加する 12

4.2 SSH でサーバーに入れないようにする 13

4.3 削除する・パスワードを変える 13

4.4 管理者の権限（clab_admins） 13

5. 日常の運用 13

5.1 サービスの状態を見る・再起動する 13

5.2 サーバーの使用状況を見る 14

5.3 ラボの状況を見る（管理者） 14

5.4 利用者のラボを止める・消す 14

5.5 画面を更新する 14

5.6 サービス（プロキシ）を更新する 14

5.7 ノードのイメージを更新する 15

5.8 API サーバーを更新する 15

6. バックアップと復旧 16

6.1 バックアップするもの 16

6.2 サーバーを再起動したとき 16

7. セキュリティ 17

8. トラブルシューティング 18

8.1 症状から調べる 18

8.2 ディスクを空ける 20

8.3 ノードの中を調べる 20

8.4 HTTPS の証明書を作り直す 20

8.5 困ったときの情報の集め方 21

付録 A　ポート・サービス一覧 21

付録 B　環境変数 22

付録 C　管理用コマンド早見表 23

1. はじめに

1.1 この文書について

Free-Networking-labs（以下「本システム」）を動かすサーバーの、**構築・運用・保守**の手順をまとめた文書です。 利用者向けの使い方は「操作マニュアル（利用者編）」、システムの設計と評価は「評価者編」を見てください。

対象は、サーバー 1 台に本システムを構築し、利用者のアカウントを管理し、障害に対応する人です。
Linux の基本的な 操作（ `sudo` 、 `systemctl` 、 `docker` ）ができることを前提にしています。

> **表記**
>
> `$` で始まるコマンドは、管理者のアカウント（本文では `labuser` ）で実行します。 `sudo` が付くものは、 管理者権限（root）が必要です。 `<...>` は、環境に合わせて置き換えてください。

1.2 システムの構成

利用者のブラウザは、**nginx（443 番）の 1 か所**にだけつなぎます。nginx が、用途ごとに裏のサービスへ振り分けます。

構成要素	役割	待ち受け	動かし方
nginx	入口。HTTPS、静的ファイル（画面）の配信、各サービスへの振り分け、ログインの回数制限	80・443	システムのサービス（`nginx`）
clab-api-server	ラボの操作 API。**Linux アカウントで認証（PAM）**し、ラボを利用者ごとに分離して管理	8090（HTTPS）	システムのサービス（`clab-api-server`）
console-proxy	ブラウザのコンソール（WebSocket）を、認証ヘッダー付きで API へ中継	127.0.0.1:8082	ユーザーサービス
ovs-helper	ルーター・スイッチの設定の保存先（`frr-config/`、`switch-config/`）を、ラボの作成前に用意	127.0.0.1:8083	ユーザーサービス
capture-proxy	パケットキャプチャ（Wireshark の画面）の中継。使われなくなったセッションの片付け	127.0.0.1:8084（HTTPS）	ユーザーサービス
EdgeShark	キャプチャの裏方（ghostwire と packetflix の 2 コンテナ）	127.0.0.1:5001	Docker Compose
Docker	ノード（ルーター・スイッチ・PC）のコンテナを動かす	—	システムのサービス（`docker`）

```
ブラウザ ──HTTPS 443──▶ nginx ─┬─ /              ▶ 画面（静的ファイル）
                               ├─ /login /api/ ▶ clab-api-server :8090 ─▶ containerlab
─▶ Docker
                               ├─ /console    ▶ console-proxy :8082 ─▶ clab-api-server
                               ├─ /ovs/       ▶ ovs-helper :8083
                               └─ /capture/   ▶ capture-proxy :8084 ─▶ clab-api-server
─▶ EdgeShark :5001
```

ノードのコンテナは 4 種類です。

ノード	イメージ	作り方
ルーター	`fnl/frr:3`（FRR 10.2.1 にサブインターフェースの自動作成と LLDP/CDP を足したもの）	`backend/frr-router/`
L2スイッチ	`fnl/l2switch:6`（Linux ブリッジ＋ Cisco 風 CLI＋ mstpd＋ lldpd）	`backend/l2switch/`
L3スイッチ	`fnl/l3switch:7`（L2スイッチ＋ FRR＋ nftables＋ dhcp-helper。ルーティング・VRRP・DHCP リレー・ACL）	`backend/l2switch/`（`Dockerfile.l3`）
PC	`alpine:3.20`（公式イメージそのまま）	—

1.3 ディレクトリとファイルの場所

場所	内容
`~/Free-Networking-labs/`（リポジトリ）	ソース一式。`backend/`（各サービス）、`frontend/`（画面）、`docs/`（文書）
`/var/lib/containerlab/labs/<ユーザー名>/<ラボID>/`	利用者のラボ。図の定義（`.clab.yml`）、画面の配置情報（`.annotations.json`）、ルーターの設定（`frr-config/`）、スイッチの設定（`switch-config/`）
`/etc/clab-api-server/clab-api-server.env`	API サーバーの設定（**JWT の秘密鍵を含む**。root だけが読める）
`/etc/nginx/conf.d/fnl.conf`	nginx の設定（リポジトリの `backend/nginx/fnl.conf` のコピー）
`/etc/nginx/ssl/fnl/`	HTTPS の証明書と秘密鍵
`/var/www/free-networking-labs/`	画面の静的ファイル（ビルドの出力）
`~/.config/systemd/user/`	ユーザーサービス（console-proxy、ovs-helper、capture-proxy）
`/usr/local/sbin/ovs-helper-frr-config-init`	ovs-helper が root 権限で呼ぶ、限定されたラッパー

2. 動作環境

項目	要件・確認した環境
OS	Ubuntu Server 24.04 LTS
カーネル	6.7 以上（スイッチの STP が、ブリッジの MST 機能を使うため）。6.8（Ubuntu 24.04 標準）で確認
CPU・メモリ	4 コア・メモリ約 12 GB（Proxmox VE 上の VM）で運用。ディスクは 57 GB
Docker	29.x で確認
Node.js	24（nvm で導入）。プロキシの実行と画面のビルドに使う
ネットワーク	イメージの取得・ビルド（GitHub・Docker のレジストリ）のため、構築時はインターネットが要る。運用時は不要

メモリの目安（実測）

サーバーが共有なので、1 ノードあたりの消費を知っておくと、人数と台数の見積もりができます。
2026-10-15 に、動いていたラボで測った値です。

ノード	メモリ	備考
PC	約 1〜3 MB	ほとんど消費しない
ルーター	約 40〜45 MB	アイドル時。FRR のデーモン（zebra・ospfd・bgpd など）を含む
L2スイッチ	約 20〜100 MB	大きい値はファイルキャッシュを含む。CPU は 1 台あたり約 1%
L3スイッチ	約 45〜125 MB の見込み	L2スイッチ＋ FRR（zebra・ospfd・bgpd など全部でコンテナ全体約 24 MiB、実測）。イメージは約 125 MB

目安：ルーター 3 台＋スイッチ 3 台＋ PC 6 台のラボ（ノード 12 台）で、約 300〜500 MB です。

> **全員が大きなラボを同時に動かすと、メモリ（12 GB）を超えます**
>
> 40 人が全員この規模を同時に動かすと、約 12〜20 GB になり、足りなくなります。実際は、全員が同時に動かすことは 少ないので、授業では次のように案内してください。ラボは **10 台程度まで**を目安にする。使い終わったら**停止**する （画面下部のサーバーの使用率が、黄色や赤のときは特に）。また、CPU は、通信を大量に流したときに使います （リンクの帯域の既定は 100 Mbps で、1 本の通信あたり約 3%。「無制限」にすると約 44%）。

3. 新しいサーバーへのセットアップ

次の順に作業します。所要は 1〜2 時間です（イメージのビルドとダウンロードを含む）。

手順 1　OS と基本のパッケージ

```
$ sudo apt-get update && sudo apt-get -y upgrade
$ sudo apt-get install -y git openssl rsync curl
```

管理者のアカウント（本文では `labuser`）を作り、`sudo` と `docker` のグループに入れます。

手順 2　Docker

Docker の公式の手順（`https://docs.docker.com/engine/install/ubuntu/`）で導入します。導入後、次を確認します。

```
$ sudo systemctl enable --now docker
$ sudo usermod -aG docker labuser      ← 入れ直し（ログインし直し）が必要
$ docker run --rm hello-world
```

> **docker グループは root と同等です**
>
> `docker` グループに入れたアカウントは、サーバーの root と同じことができます。**管理者だけ**を入れ、利用者（学生）は入れないでください。

手順 3　clab-api-server

公式のインストーラーで、サービスとして導入します（バイナリが `/usr/local/bin/clab-api-server` に入り、設定ファイルと、`clab_api`・`clab_admins` の 2 つのグループが作られます）。

```
$ curl -fsSL https://raw.githubusercontent.com/srl-labs/clab-api-server/main/
install.sh | sudo bash -s -- install
$ sudoedit /etc/clab-api-server/clab-api-server.env
```

設定ファイルで、次の項目を確認・変更します。

項目	値	理由
`API_PORT`	`8090`	nginx がこの番号に中継する
`API_USER_GROUP`	`clab_api`	利用者のグループ（API にログインできる）
`SUPERUSER_GROUP`	`clab_admins`	管理者のグループ（他人のラボも操作できる）
`TLS_ENABLE`	`true`（`TLS_AUTO_CERT=true`）	nginx は自己署名の証明書のまま中継する（`proxy_ssl_verify off`）
`CLAB_LABS_ROOT`	`/var/lib/containerlab/labs`	**この値が前提**。ovs-helper のラッパーがこのパスを固定で参照する
`JWT_EXPIRATION`	`24h`（既定）	ログインの有効期限
`JWT_SECRET`	インストーラーが乱数で生成	**秘密にする。変えると全員がログアウトされる**

```
$ sudo install -d -m 0755 /var/lib/containerlab/labs
$ sudo systemctl enable --now clab-api-server
$ systemctl status clab-api-server
$ curl -sk https://localhost:8090/health        ← 応答があれば起動している
```

手順 3b　管理者を `clab_admins` に入れる

ovs-helper のラッパー（手順 8）は、`clab_admins` グループだけが実行できます。管理者を入れます。

```
$ sudo usermod -aG clab_admins labuser
```

手順 4　Node.js とリポジトリ

```
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh | bash
$ source ~/.nvm/nvm.sh && nvm install 24
$ git clone git@github.com:Tora1203/Free-Networking-labs.git ~/Free-Networking-labs
$ cd ~/Free-Networking-labs
```

手順 5　ノードのイメージ

ルーターとスイッチのイメージは、リポジトリから**自分でビルド**します（L3スイッチは、先に作った `fnl/l2switch:10` の上に作ります）。スイッチのビルドでは、mstpd のソースを GitHub から取得するので、インターネットが要ります。

```
$ docker build -t fnl/frr:3 backend/frr-router
$ docker build -t fnl/l2switch:10 backend/l2switch                                ←
L2スイッチの土台
$ docker build -t fnl/l3switch:7 -f backend/l2switch/Dockerfile.l3 backend/l2switch ←
L3スイッチ（上の土台の上に作る）
$ docker pull alpine:3.20
$ docker pull ghcr.io/srl-labs/wireshark-vnc-docker:latest     ← キャプチャ用。必ず先に取
得しておく
```

> **Wireshark のイメージは、事前に取得してください**
>
> キャプチャの開始には 45 秒の制限があります。イメージが無い状態で始めると、取得に時間がかかって `signal: killed` で失敗します。

手順 6　EdgeShark（キャプチャの裏方）

```
$ cd ~/Free-Networking-labs/backend/edgeshark && docker compose up -d
$ curl http://127.0.0.1:5001/version        ← {"name":"packetflix", ...} が返る
```

`restart: unless-stopped` なので、サーバーを再起動しても自動で起動します。

手順 7　プロキシ 3 つ（console-proxy・ovs-helper・capture-proxy）

それぞれ、依存を入れてユーザーサービスにします。

```
$ for d in console-proxy ovs-helper capture-proxy; do (cd backend/$d && npm install);
done

# capture-proxy は HTTPS で動くので、自己署名の証明書を作る
$ cd backend/capture-proxy && mkdir -p certs
$ openssl req -x509 -newkey rsa:2048 -keyout certs/key.pem -out certs/cert.pem -days
825 -nodes \
    -subj "/CN=capture-proxy" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

各ディレクトリの `*.service.example` を `~/.config/systemd/user/` にコピーし、`<your-username>` と node のパスを 書き換えます（node のパスは `source ~/.nvm/nvm.sh && which node`）。

```
$ mkdir -p ~/.config/systemd/user
$ cp backend/console-proxy/console-proxy.service.example ~/.config/systemd/user/
console-proxy.service
$ cp backend/ovs-helper/ovs-helper.service.example       ~/.config/systemd/user/ovs-
helper.service
$ cp backend/capture-proxy/capture-proxy.service.example  ~/.config/systemd/user/
capture-proxy.service
$ (各ファイルを編集)
$ systemctl --user daemon-reload
$ systemctl --user enable --now console-proxy ovs-helper capture-proxy
$ sudo loginctl enable-linger labuser       ← ログアウトしても、サーバーの起動時から動かし続け
る
```

3 つのプロキシは、既定で **127.0.0.1 だけ**で待ち受けます（LAN からは直接届きません）。

手順 8　ovs-helper のラッパー（sudo の限定許可）

ラボのディレクトリは利用者ごとに分離されていて、管理者のアカウントでは書き込めません。ovs-helper は、「そのユーザーのラボの下に、設定の保存先を作って、所有者を本人にする」だけを行うラッパーを、sudo の限定許可で呼びます。

```
$ sudo cp backend/ovs-helper/frr-config-init.sh /usr/local/sbin/ovs-helper-frr-config-
init
$ sudo chown root:clab_admins /usr/local/sbin/ovs-helper-frr-config-init
$ sudo chmod 750 /usr/local/sbin/ovs-helper-frr-config-init
$ echo 'labuser ALL=(root) NOPASSWD: /usr/local/sbin/ovs-helper-frr-config-init' |
sudo tee /etc/sudoers.d/ovs-helper-frr-config-init
$ sudo chmod 440 /etc/sudoers.d/ovs-helper-frr-config-init
$ sudo visudo -c
```

このラッパーは、入力（ユーザー名・ラボ名・ノード名）の形式を検査し、シンボリックリンクを拒否し、既存のファイルを上書きしません（利用者が `write memory` した設定を守るため）。

手順 9　画面のビルドと nginx

```
$ cd ~/Free-Networking-labs/frontend && npm ci && npm run build

# HTTPS の証明書（自己署名）。サーバーのホスト名と IP を入れる
$ cd ../backend/nginx && mkdir -p certs
$ openssl req -x509 -newkey rsa:2048 -keyout certs/key.pem -out certs/cert.pem -days
825 -nodes \
    -subj "/CN=<ホスト名>" \
    -addext "subjectAltName=DNS:<ホスト名>,DNS:localhost,IP:<サーバーの IP>,IP:127.0.0.1"

$ bash install.sh        ← nginx の導入・設定・起動（sudo を使う。何度実行しても同じ結果）
```

`install.sh` は、nginx を入れ、証明書と設定を置き、静的ファイルの置き場所を管理者の所有にします（以後の画面の更新に sudo は要りません）。`backend/nginx/fnl.conf` の `server_name` には、すべてを受ける `_` も入っているので、ホスト名を足さなくても動きます（ホスト名で振り分けたいときだけ、書き換えます）。

手順 10　ファイアウォール

利用者に見せるのは 22（SSH、管理者用）、80、443 だけです。

```
$ sudo ufw default deny incoming
$ sudo ufw default allow outgoing
$ sudo ufw allow 22/tcp
$ sudo ufw allow 80/tcp
$ sudo ufw allow 443/tcp
$ sudo ufw enable
```

clab-api-server の 8090 番は、待ち受け先を指定できないため、この方法で外から閉じます （nginx は同じサーバーの中から 127.0.0.1 でつなぐので影響しません）。

手順 11　動作の確認

確認	コマンド・操作	期待する結果
各サービス	`systemctl is-active docker nginx clab-api-server` と `systemctl --user is-active console-proxy ovs-helper capture-proxy`	すべて `active`
待ち受け	`ss -ltn`	外向きは 22・80・443・8090 だけ。8082・8083・8084・5001 は 127.0.0.1
画面	ブラウザで `https://<ホスト名>/`	ログイン画面（初回は証明書の警告）
ログイン	利用者のアカウントでログイン	ホームが開く
通し	ルーター・スイッチ・PC を置いて「▶ 起動」→ コンソールで `ping`	通信できる
キャプチャ	線を右クリック →「パケットキャプチャ」	Wireshark のタブが開く

4. 利用者のアカウント管理

利用者は、**サーバーの Linux アカウント**です（パスワードは Linux のもの）。本システムに独自のユーザー DB はありません。

4.1 追加する

```
$ sudo useradd -m -G clab_api -s /usr/sbin/nologin <ユーザー名>
$ sudo passwd <ユーザー名>
```

- グループは **`clab_api` だけ**にします（`clab_admins`・`docker`・`sudo` に入れない）。
- シェルを `/usr/sbin/nologin` にすると、SSH でシェルが使えません。本システムのログインは PAM のパスワード認証で、シェルは見ない設計ですが、**アカウントを作ったら、一度、画面でログインできるかを試してください**（`nologin` での確認は、まだしていません。既存の利用者のアカウントは `/bin/sh` で動作を確認しています）。
- 初期パスワードを伝え、後で自分で変えてもらいます。パスワードの変更は、サーバーの `passwd` で行います （本システムの画面にはパスワード変更の機能はありません）。

4.2 SSH でサーバーに入れないようにする

利用者がサーバーにログインできないよう、`clab_api` グループの SSH を拒否します（最初の 1 回だけ設定）。

```
$ echo 'DenyGroups clab_api' | sudo tee /etc/ssh/sshd_config.d/90-fnl-deny-
students.conf
$ sudo sshd -t && sudo systemctl reload ssh
```

管理者のアカウントは `clab_api` に入れないでください（SSH できなくなります）。

4.3 削除する・パスワードを変える

```
$ sudo passwd <ユーザー名>              ← パスワードの再設定
$ sudo usermod -L <ユーザー名>          ← 一時的にロック（ログインできなくなる）
$ sudo userdel <ユーザー名>             ← 削除（ホームも消すときは -r）
```

ユーザーを消す前に、そのユーザーのラボのコンテナを止めてください（5.4）。ラボのデータは、`/var/lib/containerlab/labs/<ユーザー名>/` に残るので、不要なら削除します。

4.4 管理者の権限（`clab_admins`）

`clab_admins` のアカウントは、**他の利用者のラボも見て、操作できます**。教員や管理者だけを入れてください。

5. 日常の運用

5.1 サービスの状態を見る・再起動する

```
# システムのサービス
$ systemctl status docker nginx clab-api-server
$ sudo systemctl restart clab-api-server       ← 設定を変えたとき
$ sudo systemctl reload nginx                  ← nginx の設定を変えたとき（接続を切らない）

# ユーザーサービス（管理者のアカウントで）
$ systemctl --user status console-proxy ovs-helper capture-proxy
$ systemctl --user restart capture-proxy

# ログ
$ journalctl -u clab-api-server --since "-1h"
$ journalctl --user -u capture-proxy -n 50
$ sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

プロキシを再起動すると、そのプロキシ経由の接続（コンソールのタブ、キャプチャ）が切れます。利用者が 使っていない時間帯に行ってください（`ss -tn state established '( sport = :8082 or sport = :8084 )'` で、つながっている接続を確認できます）。

5.2 サーバーの使用状況を見る

利用者と同じく、画面下部の「サーバー」の表示（CPU・メモリ・ディスク）で、手軽に確認できます。詳しくは次のとおりです。

```
$ free -h                        ← メモリ
$ df -h /                        ← ディスク
$ docker stats --no-stream       ← ノードごとのメモリ・CPU
$ docker ps --format '{{.Names}}' | sed -E 's/^clab-(.*)-[^-]+$/\1/' | sort | uniq
-c    ← ラボごとのノード数
$ uptime                         ← load average（コア数を超えたら混んでいる）
```

5.3 ラボの状況を見る（管理者）

```
$ docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
$ ls -la /var/lib/containerlab/labs/*/                ← 利用者ごとのラボ
$ du -sh /var/lib/containerlab/labs/*                 ← 利用者ごとのディスク
```

`clab_admins` のアカウントで画面にログインしても、他の利用者のラボを見られます（4.4）。

5.4 利用者のラボを止める・消す

利用者に無断で消さないでください。確認がとれたものだけ、次のようにします。

```
$ docker stop $(docker ps -q --filter name=clab-<ラボ ID>-)          ← ラボのノードを止める
（設定は write memory したものだけ残る）
$ sudo rm -rf /var/lib/containerlab/labs/<ユーザー名>/<ラボ ID>      ← ラボの保存データを消す
（戻せない）
```

コンテナを止めたラボは、利用者の画面で「停止中」になり、「▶ 起動」で戻せます。

5.5 画面を更新する

画面（フロントエンド）の変更を反映するときは、`sudo` なしで行えます。

```
$ cd ~/Free-Networking-labs && git pull
$ bash backend/nginx/deploy-frontend.sh       ← ビルドして /var/www/free-networking-
labs に配置
```

利用者は、ブラウザを再読み込み（F5）すると、新しい画面になります。

5.6 サービス（プロキシ）を更新する

```
$ cd ~/Free-Networking-labs && git pull
$ (cd backend/capture-proxy && npm install)        ← 依存が変わったときだけ
$ systemctl --user restart console-proxy ovs-helper capture-proxy
```

`backend/nginx/fnl.conf` を変えたときは、`bash backend/nginx/install.sh` で、`/etc/nginx/conf.d/` に反映します。 ovs-helper のラッパー（`frr-config-init.sh`）を変えたときは、手順 8 の `cp` 〜 `chmod` をやり直します（sudoers はそのまま）。

5.7 ノードのイメージを更新する

ルーターやスイッチのイメージを変えたときは、タグを上げてビルドし（例：`fnl/l2switch:10`）、`frontend/src/types/lab.ts` の イメージ名を新しいタグにして、画面を更新します。

- 既存のラボは、**古いタグのまま動きます**。作り直したときに新しいタグになります。
- 画面は、古いタグのルーター（`fnl/frr:*`）とスイッチ（`fnl/l2switch*`・`fnl/l3switch*`）を、**前方一致で**ルーター・スイッチと判定します （タグが違っても、PC と間違えません）。
- 古いタグのイメージは、使われなくなってから消します（`docker rmi fnl/l2switch:6`）。使用中のコンテナがあると消せません。

> **スイッチの CLI（swcli.py）を変えたら、土台から作り直す**
>
> L3スイッチのイメージは、`fnl/l2switch:N` の上に作ります。`swcli.py` を変えたのに土台のタグを上げ忘れると、L3スイッチの中の `swcli` が古いままになり、新しい引数のチェックが無くてエラーになります。`swcli.py` を変えたときは **`fnl/l2switch` のタグを上げて作り直し、`Dockerfile.l3` の `BASE` も合わせてから**、`fnl/l3switch` を作り直してください。`l3.py`・`acl.py` だけを変えたときは、`fnl/l3switch` だけで足ります。 あわせて `frontend/src/types/lab.ts` のイメージ名を新しいタグにします（既存のラボは、開くと新しいタグに作り直されます）。

5.8 API サーバーを更新する

```
$ curl -fsSL https://raw.githubusercontent.com/srl-labs/clab-api-server/main/
install.sh | sudo bash -s -- upgrade
```

更新の後、`systemctl status clab-api-server` と、ログインと「▶ 起動」で、動作を確認してください。API の仕様が変わると、 画面が動かなくなることがあります（本システムは v0.6.0 で確認）。更新の前に、動いている版を控えておきます （`clab-api-server --version`）。戻すときは、`upgrade --version <タグ>` で古い版を入れます。

6. バックアップと復旧

6.1 バックアップするもの

対象	場所	理由
利用者のラボ	`/var/lib/containerlab/labs/`	図・配置・ルーターとスイッチの保存済みの設定。**最も重要**
API サーバーの設定	`/etc/clab-api-server/clab-api-server.env`	JWT の秘密鍵を含む
nginx の設定と証明書	`/etc/nginx/conf.d/fnl.conf`、`/etc/nginx/ssl/fnl/`	
ユーザーサービスの設定	`~/.config/systemd/user/*.service`	
ソース	GitHub のリポジトリ	変更のないものは不要

ノードのコンテナの中の状態は、バックアップ不要です（保存したい設定は `write memory` で上のラボのデータに残ります）。

```
$ sudo tar czf /backup/fnl-$(date +%F).tgz /var/lib/containerlab/labs /etc/clab-api-server /etc/nginx/conf.d/fnl.conf /etc/nginx/ssl/fnl
```

6.2 サーバーを再起動したとき

- Docker、nginx、clab-api-server は、サーバーの起動時に自動で起動します（`systemctl is-enabled` で確認）。
- ユーザーサービスは、`loginctl enable-linger` を設定していれば、起動時から自動で起動します（手順 7）。
- EdgeShark は `restart: unless-stopped` で、自動で戻ります。
- ノードのコンテナは、Docker の再起動ポリシー（`always`）で戻ります。**ただし、コンテナを再起動した後に、ノードどうしのリンクと中の設定が正しく戻るかは、まだ確認していません。**

> **再起動後の復旧は、未検証です**
>
> 本番の運用の前に、利用者のいない時間に、サーバーを再起動して、ラボが戻るかを一度試してください。 戻らなかったラボは、利用者の画面の「■ 停止」→「▶ 起動」、または「作り直す」で復旧できる見込みです （ルーターとスイッチは、`write memory` した設定が残ります）。

7. セキュリティ

詳しい点検の結果は、リポジトリの `docs/security.md` にあります。運用で守ることを、ここにまとめます。

- **公開するポートは 22・80・443 だけ**（ufw）。8090 は外から閉じる。
- **利用者のアカウント**は `clab_api` だけに入れる。`docker`・`clab_admins`・`sudo` に入れない（4.1）。SSH は拒否する（4.2）。
- **ログインの総当たり対策**：nginx が、同じ IP から 1 分に 30 回（まとめて 40 回）を超える `/login` を断ります（429）。 教室で多人数が同じ IP（NAT）から一斉にログインしても、通る設定です。
- **HTTPS**：証明書は自己署名です。有効期限（825 日）の前に、作り直して差し替えます（8.4）。
- **ノードの権限**：ルーター・スイッチ・PC のコンテナは、特権なし（`privileged: false`）で、必要最小限の権限だけ付けています （PC は `NET_ADMIN`、L2スイッチも `NET_ADMIN`、ルーターと**L3スイッチ**は FRR（zebra）の起動に `NET_RAW`・`SYS_ADMIN` も）。 Docker の標準の AppArmor と seccomp が効いています。
- **ログに残るもの**：nginx のアクセスログ（root と `adm` グループだけが読める）には、キャプチャの URL に含まれる ログイン用のトークン（24 時間有効）が残ります。ログを他の人に渡すときは、トークンを消してください。
- **ラボの分離**：他の利用者のラボは、API が所有者で絞っていて、画面にも出ません（`clab_admins` を除く）。

8. トラブルシューティング

8.1 症状から調べる

症状	原因として多いもの	確認・対処
画面が開かない（接続できない）	nginx が止まっている／ファイアウォール	`systemctl status nginx`、`sudo ufw status`
画面は開くが、ログインできない	clab-api-server が止まっている／アカウントが `clab_api` に入っていない／429（回数制限）	`systemctl status clab-api-server`、`id <ユーザー名>`、`journalctl -u clab-api-server`
画面は出るが、一覧が出ない・起動できない（502、通信エラー）	clab-api-server が止まっている	`systemctl restart clab-api-server`
一部の利用者だけ、起動も保存もできない	開発用の URL（`:5173`）で開いている／別のブラウザで、8090 番の証明書を許可していない	本番の URL（`https://<ホスト名>/`）で開くよう案内する。サーバーのログに `TLS handshake error ... unknown certificate` が並んでいれば、この状態
「古い形式のラボ」と出る	古い版で作ったラボ	利用者が「作り直す」（`write memory` していない設定は消える）
起動しない「lab '...' not found or not owned by user」	**別の利用者が同じラボ ID を使っている**（2026-10-15 より前に作ったラボどうし）	片方で新しいラボを作って、エクスポート→インポートで移す。新しいラボの ID には、ユーザー名が付くので起こらない
起動時に `bind path ... no such file or directory`	ルーター・スイッチの設定の保存先がない（ovs-helper が動いていない、sudo のラッパーがない）	`systemctl --user status ovs-helper`、`sudo -n -l` で `ovs-helper-frr-config-init` が許可されているか
ovs-helper の応答が 500 で、`sudo: a password is required`	sudoers の設定漏れ	手順 8 をやり直す
コンソールが開かない	console-proxy が止まっている／ノードが起動していない	`systemctl --user status console-proxy`
キャプチャが `503 EdgeShark not running`	EdgeShark が止まっている	`cd backend/edgeshark && docker compose up -d`
キャプチャが `signal: killed` で失敗	Wireshark のイメージが未取得（初回の取得が制限時間を超えた）	`docker pull ghcr.io/srl-labs/wireshark-vnc-docker:latest`

症状	原因として多いもの	確認・対処
キャプチャのタブを閉じても、コンテナが残る	古い仕様。今は、タブを閉じて 60 秒で、capture-proxy が片付ける	`docker ps` に `ws_vnc` が残るなら、`journalctl --user -u capture-proxy` の `[idle]` を確認
リンクの品質の反映に失敗「busy with netem operation」	同じラボに同時に品質を入れた	画面が自動で順に再試行する。続くときは、もう一度保存
ディスクがいっぱい	古いイメージ・ビルドのキャッシュ・ログ	8.2
メモリが足りない（スワップが増える）	同時に動いているノードが多い	5.2 で多いラボを探し、利用者に停止を依頼

8.2 ディスクを空ける

```
$ df -h /
$ docker system df                                ← 何が大きいか
$ docker image prune                              ← 名前のない（使われていない）イメージ
$ docker builder prune                            ← ビルドのキャッシュ
$ docker rmi fnl/l2switch:4 fnl/frr:1             ← 古いタグ（使用中だと消えない）
$ sudo journalctl --vacuum-time=14d               ← 古いシステムのログ
```

`docker system prune -a` は、使っていないイメージ（`alpine:3.20` を含む）を全部消すので、使わないでください（再取得に時間がかかります）。

8.3 ノードの中を調べる

```
$ docker exec -it clab-<ラボ ID>-<ノード名> sh           ← ノードのシェル
$ docker exec clab-<ラボ ID>-<スイッチ名> swcli          ← スイッチの CLI（標準入力から）
$ docker exec clab-<ラボ ID>-<ルーター名> vtysh -c 'show running-config'
$ docker logs clab-<ラボ ID>-<ノード名>
```

スイッチの内部の状態は、`/run/fnl-switch/`（`running.json`、ログ、`rates.json` など）にあります。設定の保存先は `/etc/fnl-switch/startup.json`（ホストの `switch-config/<名前>/` が見えている）です。

8.4 HTTPS の証明書を作り直す

証明書の期限が近づいたら、手順 9 の `openssl` を同じように実行して `bash backend/nginx/install.sh` で差し替えます。期限は、`openssl x509 -in /etc/nginx/ssl/fnl/cert.pem -noout -enddate` で確認できます（sudo が要ります）。利用者のブラウザには、また警告が 1 回出ます。キャプチャのプロキシの証明書（`backend/capture-proxy/certs/`）は、サーバーの中のやりとりだけに使うので、利用者には影響しません。

8.5 困ったときの情報の集め方

不具合を開発者に伝えるときは、次の情報があると原因が早く分かります。

1. 何をしたか、いつ（時刻）、どの利用者か
2. 画面に出たエラーの文
3. `journalctl -u clab-api-server --since "-10min"` の出力
4. `journalctl --user -u console-proxy -u ovs-helper -u capture-proxy --since "-10min"` の出力
5. `docker ps -a` と `free -h`、`df -h /` の出力

付録 A　ポート・サービス一覧

ポート	サービス	待ち受け先	外に出すか
22	SSH	全部	管理者のみ
80	nginx（443 へ転送）	全部	出す
443	nginx（HTTPS）	全部	出す
8090	clab-api-server（HTTPS）	全部（変更不可）	**ufw で閉じる**
8082	console-proxy	127.0.0.1	出さない
8083	ovs-helper	127.0.0.1	出さない
8084	capture-proxy（HTTPS）	127.0.0.1	出さない
5001	EdgeShark（packetflix）	127.0.0.1	出さない

付録 B　環境変数

サービス	変数	既定値	内容
console-proxy	`PROXY_PORT`	8082	待ち受けのポート
console-proxy	`LISTEN_HOST`	127.0.0.1	待ち受けのアドレス
console-proxy	`CLAB_API_BASE_URL`	https://localhost:8090	API サーバーの URL
ovs-helper	`OVS_HELPER_PORT`	8083	待ち受けのポート
ovs-helper	`LISTEN_HOST`	127.0.0.1	待ち受けのアドレス
ovs-helper	`CORS_ALLOWED_ORIGINS`	（空）	開発で別のオリジンから使うときだけ設定
ovs-helper	`OVS_HELPER_FRR_CONFIG_INIT_BIN`	/usr/local/sbin/ovs-helper-frr-config-init	ラッパーの場所
capture-proxy	`CAPTURE_PROXY_PORT`	8084	待ち受けのポート
capture-proxy	`LISTEN_HOST`	127.0.0.1	待ち受けのアドレス
capture-proxy	`CAPTURE_IDLE_SECONDS`	60	接続がなくなってから、キャプチャを片付けるまでの秒数
capture-proxy	`CAPTURE_PROXY_TLS_CERT` / `_KEY`	certs/cert.pem / key.pem	証明書
clab-api-server	`CAPTURE_WIRESHARK_SESSION_TTL`	2h	キャプチャの 1 回あたりの最長時間

ユーザーサービスの環境変数は、`~/.config/systemd/user/<名前>.service` の `Environment=` に書きます。

付録 C　管理用コマンド早見表

目的	コマンド
全サービスの状態	`systemctl is-active docker nginx clab-api-server; systemctl --user is-active console-proxy ovs-helper capture-proxy`
待ち受けの確認	`ss -ltn`
動いているノード	`docker ps`
メモリ・CPU	`free -h`、`docker stats --no-stream`
ディスク	`df -h /`、`docker system df`
利用者の所属グループ	`id <ユーザー名>`、`getent group clab_api clab_admins docker sudo`
画面の更新	`bash backend/nginx/deploy-frontend.sh`
プロキシの再起動	`systemctl --user restart console-proxy ovs-helper capture-proxy`
nginx の設定の確認と反映	`sudo nginx -t && sudo systemctl reload nginx`
API サーバーのログ	`journalctl -u clab-api-server --since "-1h"`